Pseudocódigos de las heurísticas de Erdoğan et al. (2012) adaptadas al TRPPD-H_3

Taller de investigación

Noviembre de 2026

Notación

- Una ruta es $\pi = (\pi_1, \ldots, \pi_n)$, con los clientes en el orden de visita; $\pi_0 = 0$ es el depósito.
- En la posición k, una operación anterior a completar la entrega retrasa al cliente π_k y a los siguientes: pesa $W_k = n - k + 1$ en $\sum_i D_i$. Una operación posterior pesa $W_k - 1$.
- $h_a = u_a + \ell_a$ y $h_b = u_b + \ell_b$. La política del cliente i es $s_i = 1$ (Política 1) o $s_i = 0$ (Política 2).
- La carga a bordo no depende de las políticas: π respeta Q si $\sum_{r \in V_c} \alpha_r + \sum_{m=1}^{k} (\beta_{\pi_m} - \alpha_{\pi_m}) \leq Q$ para todo $k \in \{0, \ldots, n\}$.
- Vecindario $\mathcal{N}(\pi)$ (Sección 3 del artículo): las rutas que respetan Q y se obtienen con $\textsc{Relocate}(\pi, i, p)$, que lleva el cliente de la posición i a la posición p, o con $\textsc{DosOpt}(\pi, i, j)$, $i < j$, que invierte $\pi_i, \ldots, \pi_j$ (cambia los arcos (π_{i-1}, π_i) y (π_j, π_{j+1}), con $\pi_{n+1} = 0$).
- Atributo tabú $a(\sigma)$ del movimiento que produce σ: el cliente v en un relocate, (v, v); las posiciones (i, j) en un 2-opt.
- Parámetros del artículo: $N_{iter} = 200$, $N_{rand} = \max\{1, \text{round}(0{,}1\, n)\}$, lista tabú de largo $L = \max\{1, \lfloor 0{,}5\, n \rfloor\}$ y $N^* = \lfloor \sqrt{N_{iter}} \rfloor = 14$ en ITS.
- *Exacto* y *heurístico* se refieren a cómo se evalúan los vecinos: con $\textsc{EvalExacta}$ (Algoritmo 2, $O(n^2)$) o con $\textsc{EvalHeur}$ (Algoritmo 4, $O(n)$). En la versión heurística el vecino elegido se vuelve a evaluar con $\textsc{EvalExacta}$, así que los costos que se comparan son siempre exactos.

1. Evaluación de una ruta

Algoritmo 1 $P(\pi, \alpha, \beta, u_b, \ell_b, h_a)$ (Algoritmo 2.1 adaptado)

Entrada: ruta π con n clientes; α_j, β_j: unidades de entrega y recogida del cliente j; u_b, ℓ_b: tiempos de descargar y cargar una recogida; h_a: tiempo de retirar y recargar una entrega.

Salida: Costos p_{ij} de aplicar la Política 1 en $i{+}1, \ldots, j{-}1$ y la Política 2 en j, $\forall\, i \in \{0, \ldots, n{-}1\}$, $j \in \{i{+}1, \ldots, n\}$.

for $i \leftarrow 0$ **to** $n-1$ **do**
  $\beta' \leftarrow 0$; $\theta \leftarrow 0$ // se aplicó la Pol. 2 en i
  $\alpha' \leftarrow \sum_{m=i+1}^{n} \alpha_{\pi_m}$ // entregas que aún quedan a bordo
  for $j \leftarrow i+1$ **to** n **do**
    $W \leftarrow n - j + 1$ // peso de latencia en la posición j
    $\alpha' \leftarrow \alpha' - \alpha_{\pi_j}$ // entregar, con Pol. 1 o 2
    $b \leftarrow \big(u_b W + \ell_b (W-1)\big)\, \beta'$ // retirar y recargar las recogidas de la puerta
    if $\beta' + \beta_{\pi_j} > 0$ **then** // costo Pol. 1 hasta aquí + costo Pol. 2
      $p_{ij} \leftarrow \theta + b + h_a\, \alpha'\, (W-1)$
    else
      $p_{ij} \leftarrow \theta + b$ // sin recogidas, la Pol. 2 equivale a la 1
    end if
    $\theta \leftarrow \theta + b$ // Política 1 en j
    $\beta' \leftarrow \beta' + \beta_{\pi_j}$ // recoger como si se usara la Pol. 1
  end for
end for

Algoritmo 2 EvalExacta(π) (Algoritmo 2.1 y DP (1)–(2))

Entrada: ruta π; datos de la instancia: c_{ij}, α_j, β_j, u_a, ℓ_a, u_b, ℓ_b y Q.

Salida: $\sum_i D_i$ mínimo de la ruta y las políticas s que lo logran; ∞ si π no respeta Q.

if π no respeta Q **then**
  return ∞
end if
$F \leftarrow \sum_{k=1}^{n} \left[W_k \left(c_{\pi_{k-1}\pi_k} + u_a \alpha_{\pi_k} \right) + (W_k - 1)\, \ell_b \beta_{\pi_k} \right]$ // viaje y operaciones obligatorias
calcular p_{ij} con el Algoritmo 1
$f(n) \leftarrow 0$
for $i \leftarrow n-1$ **downto** 0 **do** // $f(i)$: Pol. 2 aplicada en i
  $f(i) \leftarrow \min_{j \in \{i+1, \ldots, n\}} \{p_{ij} + f(j)\}$; $sig(i) \leftarrow$ el j del mínimo
end for
$s_{\pi_k} \leftarrow 1\ \forall k$; $i \leftarrow 0$ // reconstruir las políticas
while $i < n$ **do**
  $j \leftarrow sig(i)$
  if $j < n$ **and** $\sum_{m=i+1}^{j} \beta_{\pi_m} > 0$ **then**
    $s_{\pi_j} \leftarrow 0$ // Política 2 en j
  end if
  $i \leftarrow j$
end while
return $F + f(0)$, s

Algoritmo 3 $\textsc{Umbral}(i, \pi)$ (Sección 2.2, umbral 2)

Entrada: posición i de la última Política 2 ($i = 0$: depósito); ruta π; h_a, h_b.
Salida: Recogidas junto a la puerta con las que se aplica la próxima Política 2; ∞ si no conviene.

$N \leftarrow n - i$ // clientes que faltan
$\bar{\alpha} \leftarrow \frac{1}{N} \sum_{m=i+1}^{n} \alpha_{\pi_m}$; $\bar{\beta} \leftarrow \frac{1}{N} \sum_{m=i+1}^{n} \beta_{\pi_m}$ // caso especial con promedios
if $N < 2$ **or** $\bar{\beta} = 0$ **then**
    return ∞
end if
$r \leftarrow h_b \bar{\beta} / (h_a \bar{\alpha})$
$p^* \leftarrow \left\lfloor \left(1 - 2r + \sqrt{(2r-1)^2 + 8Nr}\right)/2 \right\rfloor$ // fórmula (14)
if $p^* < 2$ **then**
    return ∞ // una sola Pol. 2 al final no ahorra
end if
$k_1^* \leftarrow \max\left\{1,\ \frac{p^*-1}{2r} + \frac{N}{p^*}\right\}$ // fórmula (13) con $i = 1$
return $k_1^* \bar{\beta}$

Algoritmo 4 $\textsc{EvalHeur}(\pi)$ (evaluación lineal de la Sección 2.2)

Entrada: ruta π; datos de la instancia.
Salida: $\sum_i D_i$ de la ruta con las políticas que da el umbral, y esas políticas; ∞ si π no respeta Q.

if π no respeta Q **then**
    return ∞
end if
$F \leftarrow$ viaje y operaciones obligatorias, como en el Algoritmo 2
$\beta' \leftarrow 0$; $\theta \leftarrow 0$; $\tau \leftarrow \textsc{Umbral}(0, \pi)$
for $j \leftarrow 1$ **to** n **do**
    $W \leftarrow n - j + 1$
    $\theta \leftarrow \theta + \left(u_b W + \ell_b (W-1)\right) \beta'$ // retirar y recargar las recogidas de la puerta
    if $j < n$ **and** $\beta' + \beta_{\pi_j} > 0$ **and** $\beta' + \beta_{\pi_j} \geq \tau$ **then**
        $\theta \leftarrow \theta + h_a (W-1) \sum_{m=j+1}^{n} \alpha_{\pi_m}$ // Política 2 en j
        $s_{\pi_j} \leftarrow 0$; $\beta' \leftarrow 0$; $\tau \leftarrow \textsc{Umbral}(j, \pi)$ // nuevo umbral
    else
        $s_{\pi_j} \leftarrow 1$; $\beta' \leftarrow \beta' + \beta_{\pi_j}$ // Política 1 en j
    end if
end for
return $F + \theta$, s

2. Heurística de 2 fases

Algoritmo 5 $\mathrm{TSP}(V, c)$ (fase 1)

Entrada: nodos $V = \{0, \ldots, n\}$ (0: depósito); tiempos de viaje c_{ij}.
Salida: Ciclo hamiltoniano C por el depósito y los clientes, de largo $L(C) = \sum_{(i,j)\in C} c_{ij}$.

$C \leftarrow \emptyset$; $L(C) \leftarrow \infty$
for all $v \in V$ **do** // el artículo usa Lin–Kernighan
  $C' \leftarrow$ ciclo del vecino más cercano que parte en v
  repeat // primera mejora
    $mejora \leftarrow$ **false**
    if invertir un tramo de C' (2-opt) reduce $L(C')$ **then**
      aplicar ese 2-opt; $mejora \leftarrow$ **true**
    else if mover un tramo de 1 a 3 nodos, en cualquier sentido (Or-opt), reduce $L(C')$ **then**
      aplicar ese Or-opt; $mejora \leftarrow$ **true**
    end if
  until not $mejora$
  if $L(C') < L(C)$ **then**
    $C \leftarrow C'$
  end if
end for
return C

Algoritmo 6 $\textsc{DosFases}(V, c)$ (TSP + Algoritmo 2.1)

Entrada: instancia del TRPPD-H_3: nodos V, tiempos c_{ij} y los datos de $\textsc{EvalExacta}$.
Salida: Soluciones iniciales π^{tsp} y π^{inv}, una por sentido del ciclo TSP; la mejor de las dos es la solución de la heurística de 2 fases.

$C \leftarrow \mathrm{TSP}(V, c)$ // fase 1
$\sigma^{\mathrm{tsp}} \leftarrow$ clientes de C en su orden, sin el depósito
$\sigma^{\mathrm{inv}} \leftarrow \sigma^{\mathrm{tsp}}$ en orden inverso
for all $d \in \{\mathrm{tsp}, \mathrm{inv}\}$ **do**
  $(\sigma_1, \ldots, \sigma_n) \leftarrow \sigma^d$; $z^d \leftarrow \infty$
  for $r \leftarrow 0$ **to** $n-1$ **do** // fase 2: reubicar el depósito
    $\pi \leftarrow (\sigma_{r+1}, \ldots, \sigma_n, \sigma_1, \ldots, \sigma_r)$ // el depósito va antes de σ_{r+1}
    $z \leftarrow \textsc{EvalExacta}(\pi)$ // ∞ si no respeta Q
    if $z < z^d$ **then**
      $\pi^d \leftarrow \pi$; $z^d \leftarrow z$
    end if
  end for
end for
return $\pi^{\mathrm{tsp}}, \pi^{\mathrm{inv}}$

3. Metaheurísticas

Algoritmo 7 PERTURBAR(π, N_{rand})

Entrada: ruta π; N_{rand}: número de movimientos al azar.
Salida: Ruta perturbada que respeta Q.

for $r \leftarrow 1$ **to** N_{rand} **do**
  repeat // a lo más 100 intentos
    elegir $i, j \in \{1, \ldots, n\}$ al azar
    if $i = j$ **then**
      elegir $p \in \{1, \ldots, n\} \setminus \{i\}$ al azar; $\pi' \leftarrow \text{RELOCATE}(\pi, i, p)$
    else
      $\pi' \leftarrow \text{DOSOPT}(\pi, \text{mín}\{i, j\}, \text{máx}\{i, j\})$
    end if
  until π' respeta Q
  if π' respeta Q **then**
    $\pi \leftarrow \pi'$
  end if
end for
return π

Algoritmo 8 ILS-EXACTO$(\pi^0, N_{iter}, N_{rand})$ (Algoritmo 4.2)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}: iteraciones; N_{rand}: movimientos al azar por perturbación.
Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$, z^*.

$\pi^* \leftarrow \pi^0$; $z^* \leftarrow \text{EVALEXACTA}(\pi^0)$
for $it \leftarrow 1$ **to** N_{iter} **do**
  $\pi \leftarrow \text{PERTURBAR}(\pi^*, N_{rand})$; $z \leftarrow \text{EVALEXACTA}(\pi)$
  repeat // búsqueda local con mejor mejora
    $\pi' \leftarrow \arg\text{mín}_{\sigma \in \mathcal{N}(\pi)} \text{EVALEXACTA}(\sigma)$ // todos los vecinos con el DP
    $z' \leftarrow \text{EVALEXACTA}(\pi')$ // ya calculado al explorar
    $mejora \leftarrow (z' < z)$
    if $mejora$ **then**
      $\pi \leftarrow \pi'$; $z \leftarrow z'$
    end if
  until not $mejora$
  if $z < z^*$ **then** // se conserva la mejor ruta
    $\pi^* \leftarrow \pi$; $z^* \leftarrow z$
  end if
end for
return π^*, z^*

Algoritmo 9 ILS-HEUR$(\pi^0, N_{iter}, N_{rand})$ (Algoritmo 4.2 con evaluación heurística)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}: iteraciones; N_{rand}: movimientos al azar por perturbación.
Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$ exacto, z^*.

$\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
for $it \leftarrow 1$ **to** N_{iter} **do**
  $\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
  repeat // búsqueda local con mejor mejora
    $\pi' \leftarrow \arg\min_{\sigma \in \mathcal{N}(\pi)}$ EVALHEUR(σ) // todos los vecinos con el umbral
    $z' \leftarrow$ EVALEXACTA(π') // reevaluar solo el vecino elegido
    $mejora \leftarrow (z' < z)$
    if $mejora$ **then**
      $\pi \leftarrow \pi'$; $z \leftarrow z'$
    end if
  until not $mejora$
  if $z < z^*$ **then**
    $\pi^* \leftarrow \pi$; $z^* \leftarrow z$
  end if
end for
return π^*, z^*

Algoritmo 10 ITS-EXACTO$(\pi^0, N_{iter}, N_{rand})$ (Algoritmos 4.3 y 4.1)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}; N_{rand}: movimientos al azar por perturbación.
Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$, z^*.

$N^* \leftarrow \lfloor \sqrt{N_{iter}} \rfloor$; $L \leftarrow \max\{1, \lfloor n/2 \rfloor\}$ // iteraciones y largo de la lista tabú
$\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
for $it \leftarrow 1$ **to** N^* **do**
  $\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
  $\pi^b \leftarrow \pi$; $z^b \leftarrow z$; $T \leftarrow ()$ // búsqueda tabú desde π
  for $t \leftarrow 1$ **to** N^* **do**
    $\mathcal{A} \leftarrow \{\sigma \in \mathcal{N}(\pi) : a(\sigma) \notin T \ \vee \ \text{EVALEXACTA}(\sigma) < z^b\}$ // no tabú o aspiración
    if $\mathcal{A} = \emptyset$ **then**
      break
    end if
    $\pi \leftarrow \arg\min_{\sigma \in \mathcal{A}}$ EVALEXACTA(σ); $z \leftarrow$ EVALEXACTA(π) // se mueve aunque empeore
    agregar $a(\pi)$ al final de T; si $|T| > L$, quitar el más antiguo
    if $z < z^b$ **then**
      $\pi^b \leftarrow \pi$; $z^b \leftarrow z$
    end if
  end for
  if $z^b < z^*$ **then**
    $\pi^* \leftarrow \pi^b$; $z^* \leftarrow z^b$
  end if
end for
return π^*, z^*

Algoritmo 11 ITS-HEUR$(\pi^0, N_{iter}, N_{rand})$ (Algoritmos 4.3 y 4.1 con evaluación heurística)

Entrada: solución inicial π^0 de la heurística de 2 fases; N_{iter}; N_{rand}: movimientos al azar por perturbación.
Salida: Mejor ruta encontrada π^* y su $\sum_i D_i$ exacto, z^*.

$N^* \leftarrow \lfloor\sqrt{N_{iter}}\rfloor$; $L \leftarrow \max\{1, \lfloor n/2 \rfloor\}$
$\pi^* \leftarrow \pi^0$; $z^* \leftarrow$ EVALEXACTA(π^0)
for $it \leftarrow 1$ **to** N^* **do**
  $\pi \leftarrow$ PERTURBAR(π^*, N_{rand}); $z \leftarrow$ EVALEXACTA(π)
  $\pi^b \leftarrow \pi$; $z^b \leftarrow z$; $T \leftarrow ()$
  for $t \leftarrow 1$ **to** N^* **do**
    $\mathcal{A} \leftarrow \{\sigma \in \mathcal{N}(\pi) : a(\sigma) \notin T \ \vee$ EVALHEUR$(\sigma) < z^b\}$ // aspiración con el umbral
    if $\mathcal{A} = \emptyset$ **then**
      break
    end if
    $\pi \leftarrow \arg\min_{\sigma \in \mathcal{A}}$ EVALHEUR(σ) // vecinos con el umbral
    $z \leftarrow$ EVALEXACTA(π) // reevaluar solo el movimiento elegido
    agregar $a(\pi)$ al final de T; si $|T| > L$, quitar el más antiguo
    if $z < z^b$ **then**
      $\pi^b \leftarrow \pi$; $z^b \leftarrow z$
    end if
  end for
  if $z^b < z^*$ **then**
    $\pi^* \leftarrow \pi^b$; $z^* \leftarrow z^b$
  end if
end for
return π^*, z^*

Algoritmo 12 RESOLVER(M, N_{iter}, N_{rand}) (versión 2dir)

Entrada: método $M \in \{$ILS-EXACTO, ILS-HEUR, ITS-EXACTO, ITS-HEUR$\}$; N_{iter}; N_{rand}.
Salida: Ruta π^*, políticas s y $\sum_i D_i$ de la mejor solución.

$(\pi^{\text{tsp}}, \pi^{\text{inv}}) \leftarrow$ DOSFASES(V, c)
$z^* \leftarrow \infty$
for all $\pi^0 \in \{\pi^{\text{tsp}}, \pi^{\text{inv}}\}$ **do** // una corrida desde cada sentido
  $(\pi, z) \leftarrow M(\pi^0, N_{iter}, N_{rand})$
  if $z < z^*$ **then**
    $\pi^* \leftarrow \pi$; $z^* \leftarrow z$
  end if
end for
$(z^*, s) \leftarrow$ EVALEXACTA(π^*) // políticas óptimas de la ruta final
return π^*, s, z^*

En la heurística de 2 fases no hay búsqueda: su solución es la mejor entre π^{tsp} y π^{inv}, con las políticas que da EVALEXACTA. Los movimientos al azar usan un generador con semilla fija por instancia y algoritmo, de modo que cada corrida se puede repetir.